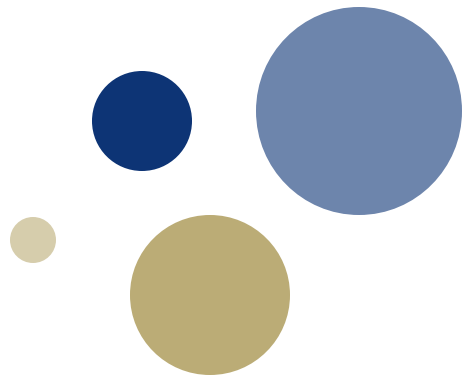




NTNU

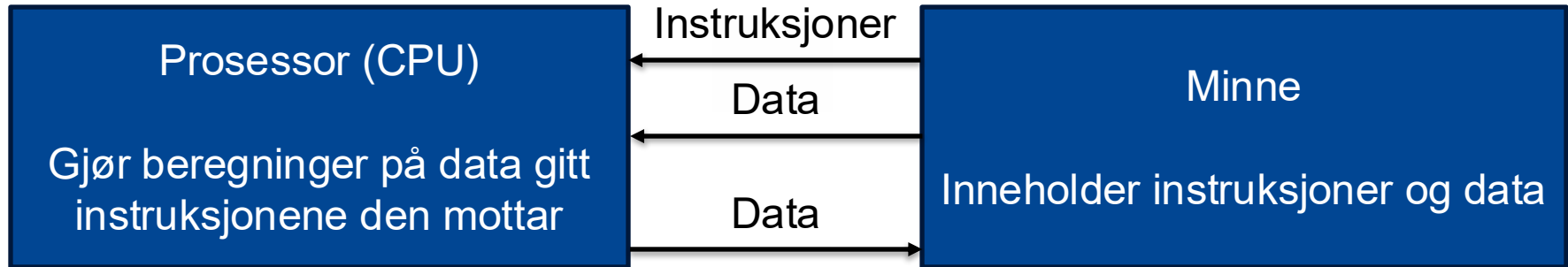
Kunnskap for en bedre verden



Forelesning 9: Høyytelses- prosessorer og minnehierarki

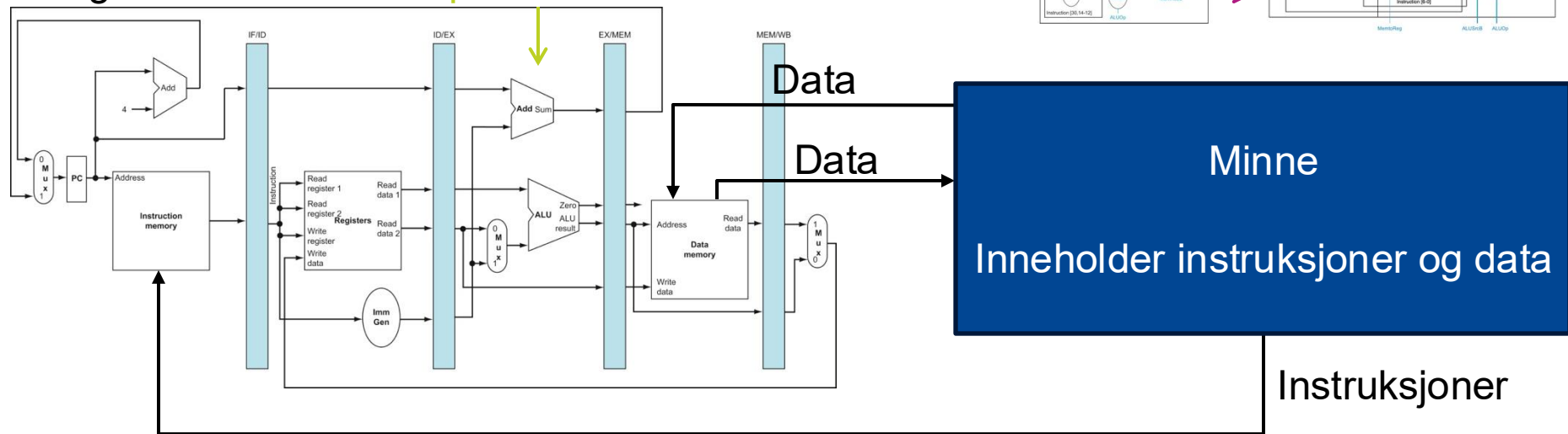
TDT4160 Datamaskiner

Prinsippet om lagrede program



Prinsippet om lagrede program

Nå kan vi lage enkeltsykelprosessorer,
flersykelprosessorer
og enkle samlebåndprosessorer



... og nå skal vi se hvordan prinsippene for hvordan vi kan oppnå (mye) høyere prosessorytelse før vi går løs på minnesystemet



Tema 5.3: Prosessorer med høyere ytelse (Kapittel 4.11)

SUPERSKALARITET

Læringsutbytte T5.3

- Studenten skal kunne beskrive hvordan en prosessor utnytter **parallellitet i tid** og **parallellitet i rom**.
- Studenten skal kunne gjenkjenne og beskrive **RAW, WAW og WAR farer** og avhengighetene som skaper dem.
- Studenten skal forstå prinsippene bak hvordan «**register renaming**» fjerner WAW og WAR farer.
- Studenten skal kjenne til fordeler og ulemper med «**static multi-issue**» prosessorer.
- Studenten skal kjenne til prinsippene bak hvordan prosessorer som utfører instruksjoner ut av rekkefølge («out of order») implementeres («dynamic multi-issue»).
- Studenten skal kunne forklare begrepet «speculation».

5-stegs samlebåndet vårt har noen utfordringer



- Vi har antatt at alle R-type instruksjoner tar like lang tid
 - ... men heltallsaddisjon krever mye enklere maskinvare enn en flyttallsaddisjon og kan dermed ta kortere tid
 - ... og klokkefrekvensen er avhengig av den tregeeste operasjonen
- Vi kan maksimalt utføre en instruksjon per klokkesykel (maksimal IPC = 1)
- Vi utfører instruksjonene i den rekkefølgen de er i programmet
 - ... og da går vi glipp av mye parallellitet

5-stegs samlebåndet vårt har noen utfordringer



- Vi har antatt at alle R-type instruksjoner tar like lang tid
 - ... men heltallsaddisjon krever mye enklere maskinvare enn en flyttallsaddisjon og kan dermed ta kortere tid
 - ... og klokkefrekvensen er avhengig av den tregeeste operasjonen
- Vi kan maksimalt utføre en instruksjon per klokkesykel (maksimal IPC = 1)
- Vi utfører instruksjonene i den rekkefølgen de er i programmet
 - ... og da går vi glipp av mye parallellitet

Høyere ytelse → Mer parallellitet

- Samlebånd utnytter parallellitet i tid
 - Hvert steg jobber med ulike instruksjoner
 - Vi kan oppnå høyere ytelse gjennom å gjøre samlebåndet dypere
 - Krever lite ekstra maskinvare
- Superskalar utføring utnytter parallellitet i rom
 - Et skritt jobber med flere instruksjoner samtidig
 - Krever mye ekstra maskinvare (for eksempel en ekstra ALU)
- Superskalare samlebånd utnytter parallellitet i tid og rom

Et superskalart samleband



Flere dataavhengigheter og datafarer

- Vi har så langt sett på sanne dataavhengigheter
 - En instruksjon leser det en annen skriver
 - Gir «Read-After-Write (RAW)» farer
- Når vi utfører flere instruksjoner samtidig, må vi også håndtere navneavhengigheter:
 - Ut-avhengigheter som gir «Write-After-Write (WAW)» farer
 - Anti-avhengigheter som gir «Write-After-Read (WAR)» farer

RAW

```
add  x3, x2, x1
sub  x7, x3, x6
```

WAW

```
add  x3, x2, x1
sub  x3, x5, x6
```

WAR

```
add  x3, x2, x1
sub  x2, x3, x6
```

WAW eksempel

WAW

mul	x3,	x2,	x1
	↓		
sub	x3,	x5,	x6

Multiplikasjon tar 3 sykler
Subtraksjon tar 1 sykel

	Sykel 1	Sykel 5	Sykel 7
x1	5		
x2	4		
x3			
x5	20		
x6	10		

«Register renaming»

- Navneavhengigheter oppstår fordi vi har et begrenset antall registre
 - ... men vi har allerede gjort så mange registre som vi kan tilgjengelig for programmet da vi laget instruksjonssettet
- Løsning: «Register Renaming»
 - Vi legger til (vesentlig) flere fysiske registre i prosessoren enn de programmet kan se (arkitekturregistre)
 - Vi tilordner fysiske registre til instruksjoner slik at vi
 - Respekterer sanne dataavhengigheter (RAW-farer)
 - Fjerner navneavhengigheter (WAW og WAR-farer)
- Nøkkelobservasjon: Hver instruksjon skriver til maksimalt ett register



add x3, x2, x1

add x3, x5, x4

add x5, x2, x4

sub x7, x3, x5

«Register renaming»

Hver instruksjon får et nytt fysisk målregister

WAW-faren er borte!

add x3, x2, x1

add x3, x5, x4

add x5, x2, x4

sub x7, x3, x5

add p6, p2, p1

add p7, p4, p3

add p8, p2, p3

sub p9, p7, p8

WAR-faren er borte!

Kun RAW-farer er igjen

TDT4255

Tilordningstabell

x1 → p1

x2 → p2

x4 → p3

x5 → p4

x3 → p6 (Gammel)

x3 → p7

x5 → p8

x7 → p9

Kompilatoren kan også finne uavhengige instruksjoner



- Ide: Kompilatoren kan generere programmet på en måte som forteller prosessoren om instruksjonene kan utføres samtidig
 - Instruksjonspakke: Alle instruksjoner i samme pakke kan utføres samtidig
 - «Very Long Instruction Word (VLIW)» og «Static multi-issue»
- Rene VLIW prosessorer er ikke vanlige
 - 👍 Maskinvaren blir enklere (se figur 4.70 i boka)
 - 👎 Kompilatoren må jobbe hardt for å utnytte instruksjonspakkene godt
 - 👎 Det er en del parallellitet som kun oppstår i kjøretid (forgreningsprediksjon og spekulativ minneaksess)

Eksempel: Et superskalart samle- bånd med «Register renaming»





Tema 5.3: Prosessorer med høyere ytelse (Kapittel 4.11)

UT-AV-REKKEFØLGE PROSESSORER

Læringsutbytte T5.3

- Studenten skal kunne beskrive hvordan en prosessor utnytter parallellitet i tid og parallellitet i rom.
- Studenten skal kunne gjenkjenne og beskrive RAW, WAW og WAR farer og avhengighetene som skaper dem.
- Studenten skal forstå prinsippene bak hvordan «register renaming» fjerner WAW og WAR farer.
- Studenten skal kjenne til fordeler og ulemper med «static multi-issue» prosessorer.
- Studenten skal kjenne til prinsippene bak hvordan **prosessorer** som utfører **instruksjoner ut av rekkefølge («out of order»)** implementeres (**«dynamic multi-issue»**).
- Studenten skal kunne forklare begrepet **«speculation»**.

Og fra T5.2:

- Studenten skal kjenne til prinsippene bak hvordan **presise unntak** implementeres i prosessorer som utfører instruksjoner **ut-av-rekkefølge**.

5-stegs samlebåndet vårt har noen utfordringer



- Vi har antatt at alle R-type instruksjoner tar like lang tid
 - ... men heltallsaddisjon krever mye enklere maskinvare enn en flyttallsaddisjon og kan dermed ta kortere tid
 - ... og klokkefrekvensen er avhengig av den tregeeste operasjonen
- Vi kan maksimalt utføre en instruksjon per klokkesykel (maksimal IPC = 1)
- Vi utfører instruksjonene i den rekkefølgen de er i programmet
 - ... og da går vi glipp av mye parallellitet

Utføringsrekkefølge

- Vi har så langt utført instruksjoner i den rekkefølgen de står i programmet
- Fundamental begrensning:
 - Når en instruksjon må stoppe, stopper alle andre instruksjoner også!
 - ... og jo lengre tid det tar å utføre en instruksjon, jo mer kunne vi ha gjort i mellomtiden
- Løsning: Ut-av-rekkefølge utføring
 - Bruker «register renaming» til å fjerne alle WAR og WAW-farer
 - Forutsetter «dynamic issue»

add p3, p2, p1

add p6, p5, p4

sub p7, p3, p6

lw p8, 0(p7)

add p11, p10, p9

add p14, p13, p12

sub p15, p11, p14

Fargene angir RAW-farer

Utføringsrekkefølge

add p3, p2, p1

add p6, p5, p4

sub p7, p3, p6

lw p8, 0(p7)

add p11, p10, p9

add p14, p13, p12

sub p15, p11, p14



Utføringsrekkefølge

Antar at maskinen kan utføre 3 instruksjoner samtidig

add	p3,	p2,	p1	add	p6,	p5,	p4	add	p11,	p10,	p9
sub	p7,	p3,	p6					add	p14,	p13,	p12
lw	p8,	0 (p7)						sub	p15,	p11,	p14



Vi har frikoblet utføringen av denne gruppen instruksjoner fra load-instruksjonen

«Data flow limit» = Det antallet instruksjoner vi kan utføre samtidig når vi kun tar hensyn til sanne dataavhengigheter

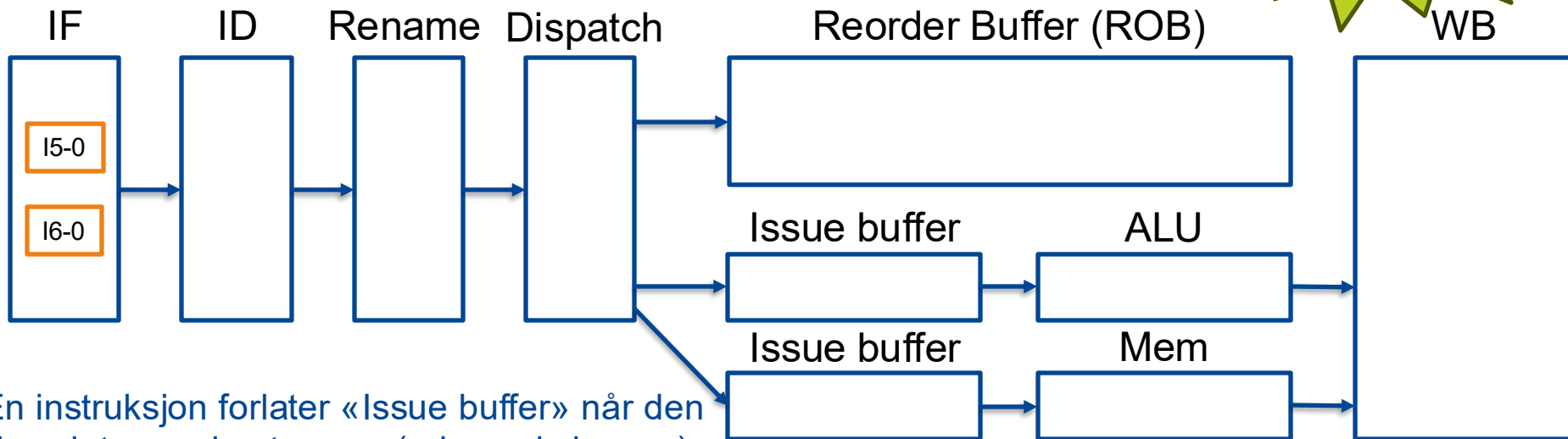
Ut-av-rekkefølge prosessorer når «data flow limit» når de har nok maskinvareressurser tilgjengelig

Eksempel: En ut-av-rekkefølge prosessor

«Issue»-steget er delt opp i «Dispatch» og et «Issue buffer» per funksjonell enhet

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw  x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1      #Legg til konstant
8     sw  x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4      #Oppdater adressen
11    j  lokke
12 slutt:
13    nop
```

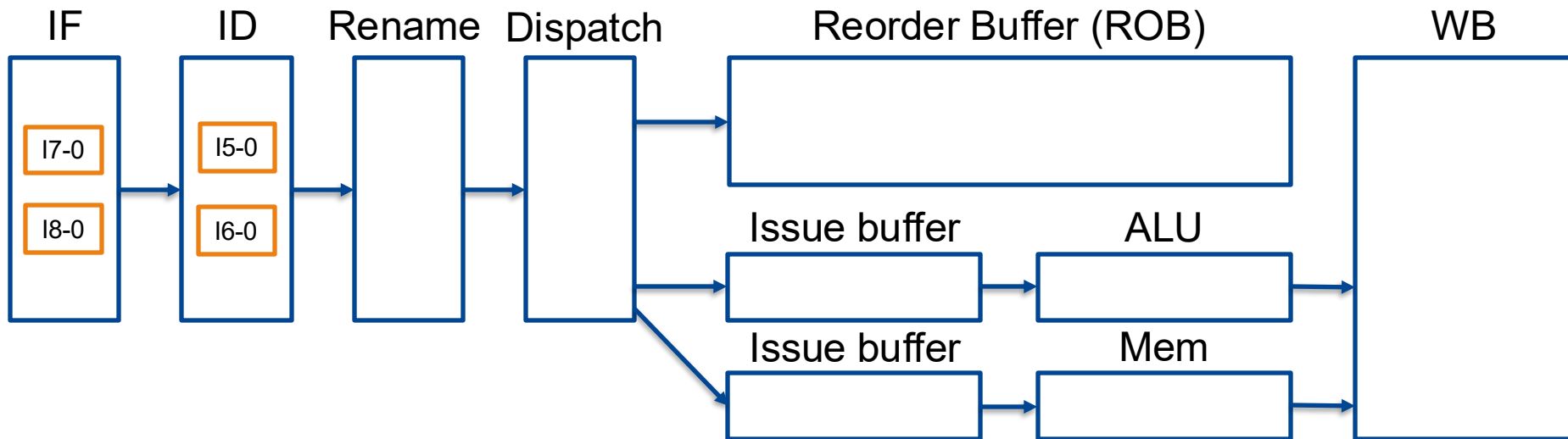
TDT4255



En instruksjon forlater «Issue buffer» når den har dataene den trenger («dynamic issue»)

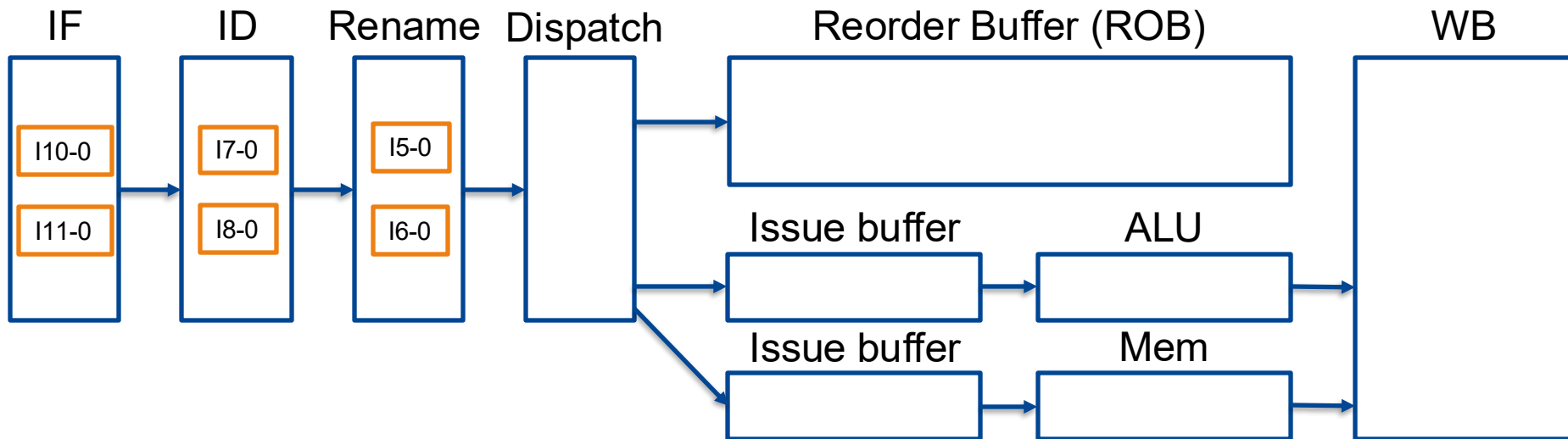
Eksempel: En ut-av- rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1    #Legg til konstant
8     sw x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4     #Oppdater adressen
11    j lokke
12 slutt:
13    nop
```



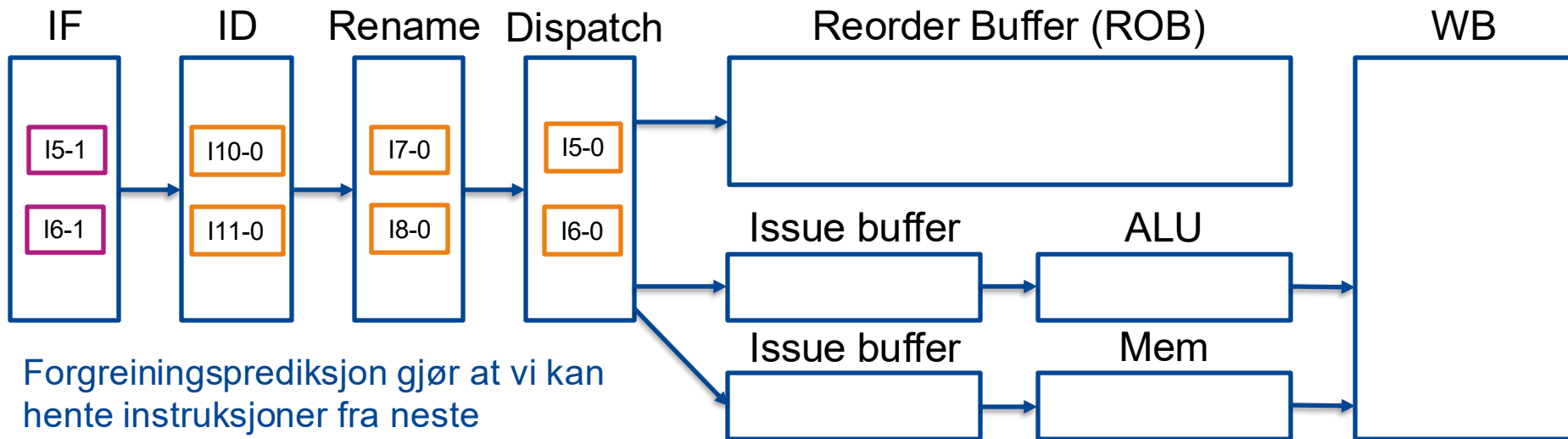
Eksempel: En ut-av- rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw  x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1     #Legg til konstant
8     sw  x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4      #Oppdater adressen
11    j  lokke
12 slutt:
13    nop
```



Eksempel: En ut-av-rekkefølge prosessor

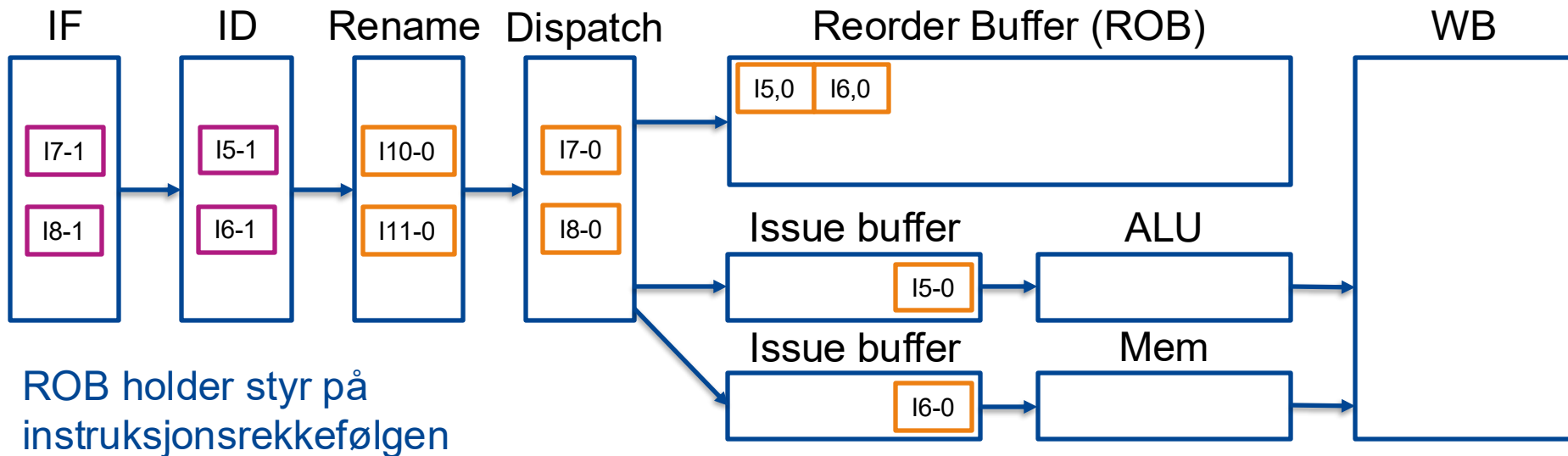
```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1    #Legg til konstant
8     sw x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4     #Oppdater adressen
11    j lokke
12 slutt:
13    nop
```



Forgreinsningsprediksjon gjør at vi kan hente instruksjoner fra neste iterasjon av løkka

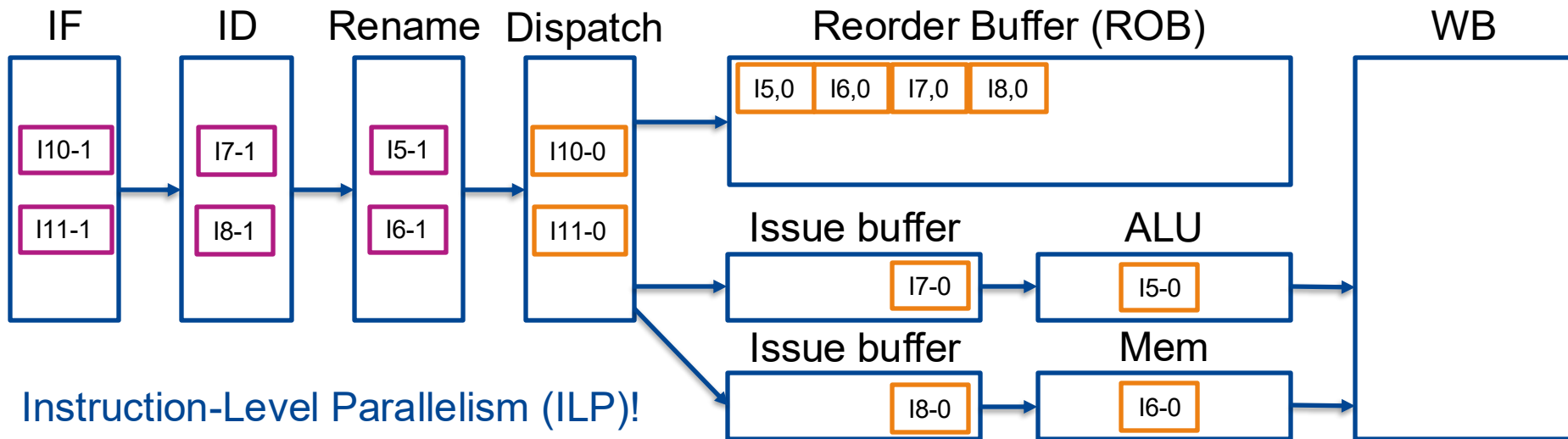
Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw  x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1     #Legg til konstant
8     sw  x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4      #Oppdater adressen
11    j  lokke
12 slutt:
13    nop
```



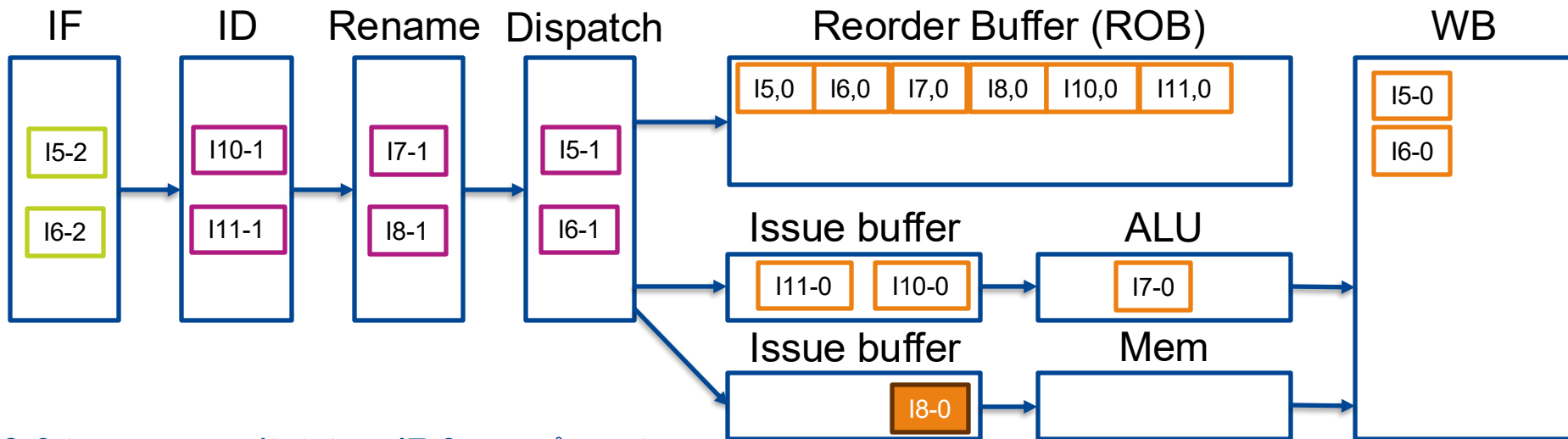
Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1    #Legg til konstant
8     sw x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4     #Oppdater adressen
11    j lokke
12 slutt:
13    nop
```



Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw  x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1     #Legg til konstant
8     sw  x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4      #Oppdater adressen
11    j  lokke
12 slutt:
13    nop
```

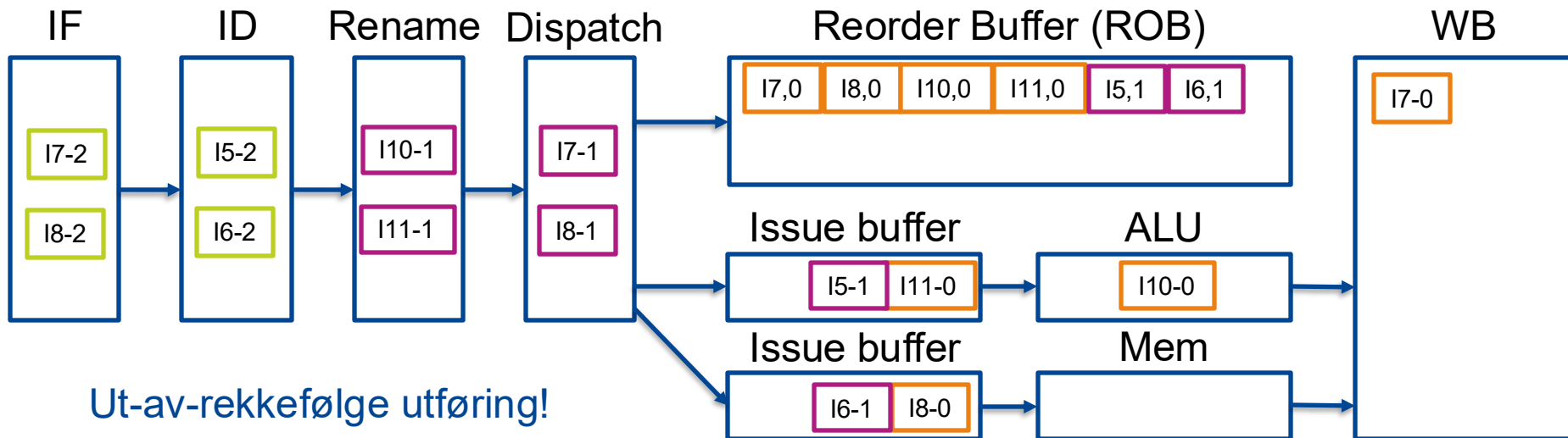


I8-0 trenger resultatet av I7-0 og må vente

Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw  x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1      #Legg til konstant
8     sw  x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4      #Oppdater adressen
11    j  lokke
12 slutt:
13    nop
```

I5,0	I5-0
I6,0	I6-0

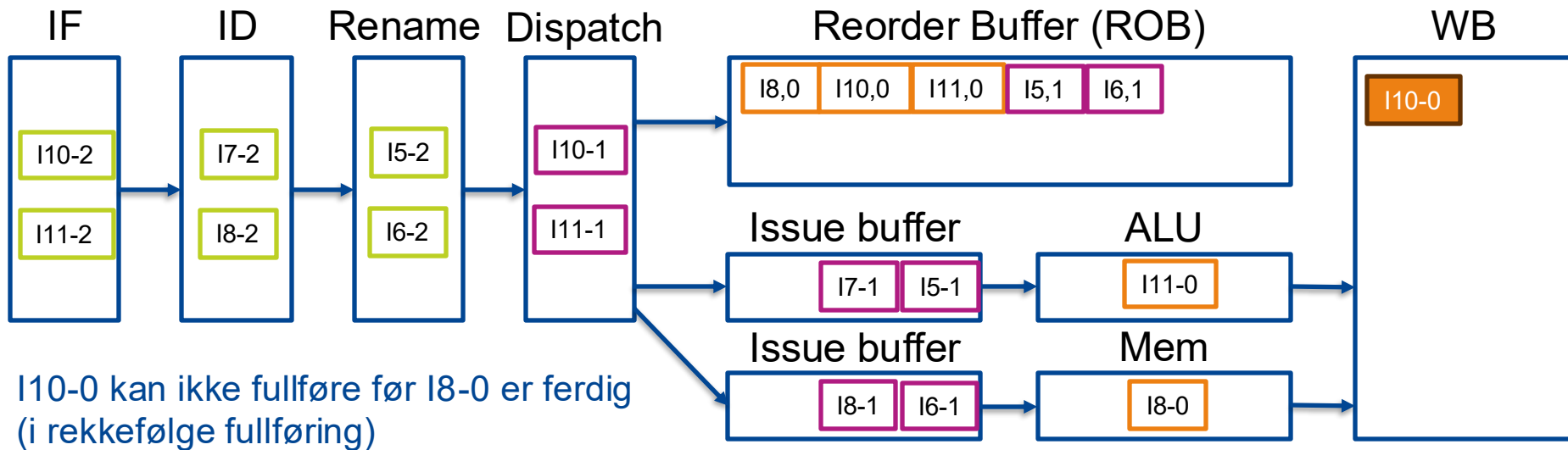


Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1     #Legg til konstant
8     sw x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4     #Oppdater adressen
11    j lokke
12 slutt:
13    nop
```

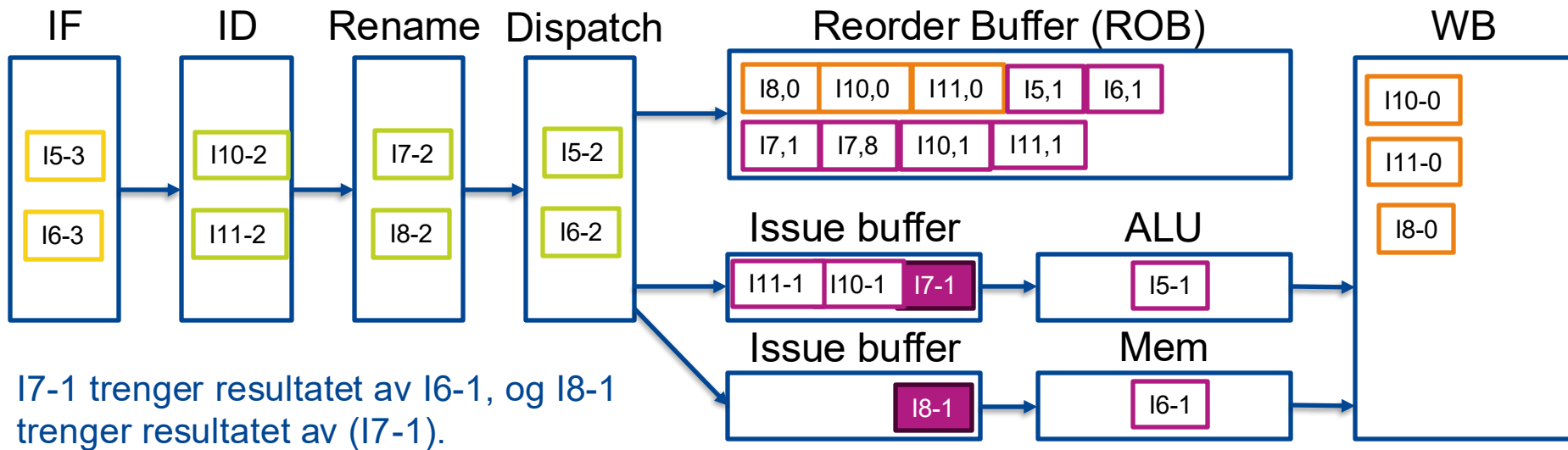
I7,0

I7-0



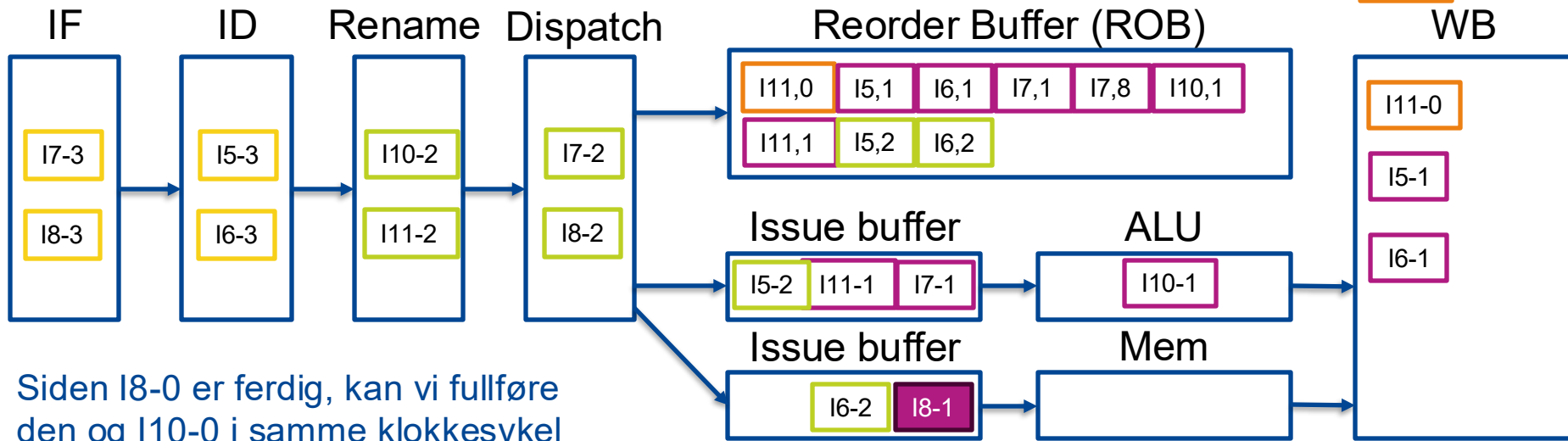
Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1    #Legg til konstant
8     sw x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4     #Oppdater adressen
11    j lokke
12 slutt:
13    nop
```



Eksempel: En ut-av-rekkefølge prosessor

```
1 addi x1, x0, 1024
2 addi x2, x0, 1064 #1024 + 10*4
3
4 lokke:
5     beq x1, x2, slutt
6     lw x3, 0(x1)      #Hent tabellverdi
7     addi x3, x3, 1    #Legg til konstant
8     sw x3, 0(x1)      #Skriv verdien tilbake
9
10    addi x1, x1, 4     #Oppdater adressen
11    j lokke
12 slutt:
13    nop
```



Siden I8-0 er ferdig, kan vi fullføre den og I10-0 i samme klokkesykel

Presise unntak og spekulasjon

- Vi må kunne fjerne instruksjoner fra samlebåndet helt frem til vi vet at de skal utføres:
 - Alle tidligere forgreningsinstruksjoner må være utført
 - Ingen tidligere instruksjoner kan ha generert unntak
- «Speculation»: Prosessoren utfører instruksjoner som den ikke er sikker på at den skal utføre
- «Re-Order Buffer (ROB)» og «Register Renaming» samarbeider om å implementere presise unntak
 - ROB holder styr på rekkefølgen
 - «Register Renaming» holder styr på verdiene



TDT4255



(Kapittel 2 til 4)

OPPSUMMERING: INSTRUKSJONER OG PROSESSORER

Hva kan vi nå?

Vi vet mye om instruksjoner

- Hvordan uttrykkes et høy-nivå program med instruksjoner?
- Hvorfor er instruksjonene slik som de er?

Vi kjenner til de fundamentale komponentene i prosessorer

- ALUer, registerfil, registre, kontrollenhet, etc.

Vi kan sette sammen de fundamentale komponentene til enkle prosessorer

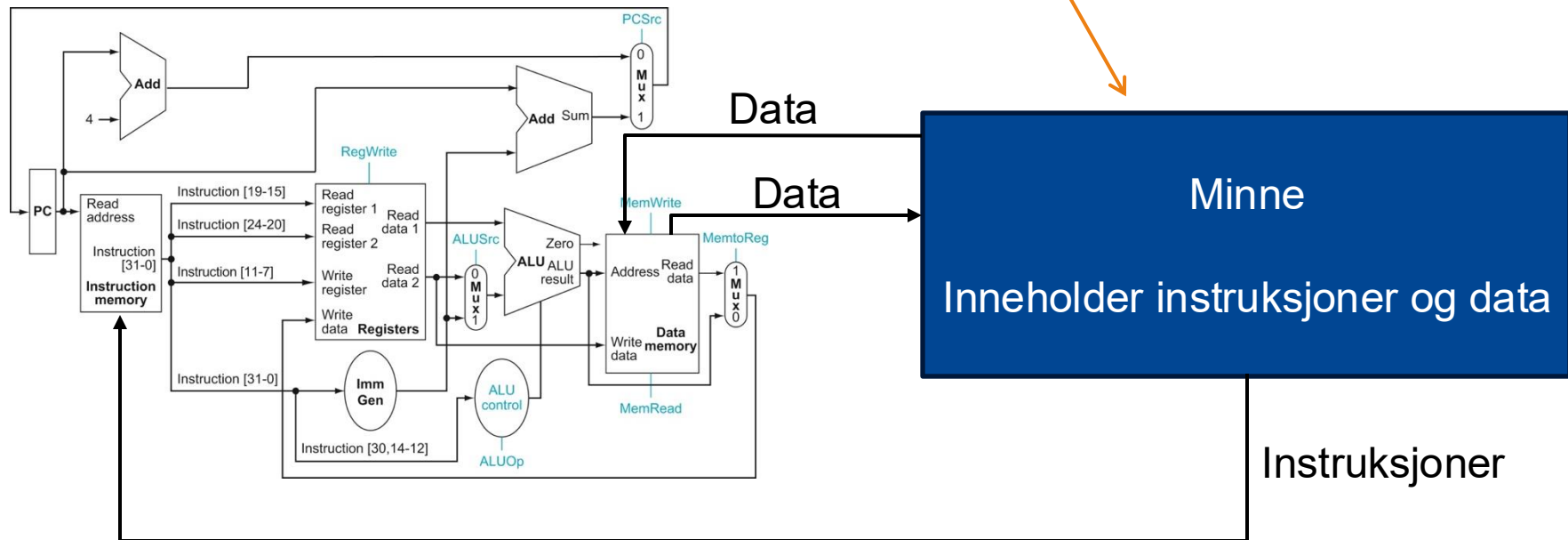
- Enkeltskykel, flersykel, og ulike samlebåndsarkitekturer

Vi kjenner til prinsippene bak moderne høy-ytelsesprosessorer

- Ut av rekkefølge instruksjonsutføring («Data-flow execution»)
- Instruksjonene fullfører i rekkefølge for å støtte presise unntak

Prinsippet om lagrede program

Nå skal vi finne ut hvordan denne virker



Med andre ord: Nå vet vi alt vi trenger å vite om prosessorer



Tema 6.1: Minnehierarki og hurtigbuffer (Kapittel 5.1 og 5.3)

MINNEHIERARKI OG HURTIGBUFFER

Læringsutbytte T6.1

- Studenten skal kunne forklare begrepene **lokalitet i tid** og **lokalitet i rom** og vite hvorfor lokalitet oppstår.
- Studenten skal forstå hvordan og hvorfor et **hierarki av minner** kan gi illusjonen av ett stort og raskt minne.
- Studenten skal forstå hvordan vi **konstruerer et direktetilordnet hurtigbuffer** (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord.
- Studenten kunne forklare hvordan **hurtigbuffer** kan **integreres** i en **samlebåndsarkitektur**.
- Studenten skal kjenne til prinsippene for hvordan hurtigbufferer håndterer **skriveoperasjoner**.
- Studenten skal kunne **beregne** hvordan **minneaksesstid** påvirker datamaskinens **ytelse**.
- Studenten skal kunne **beregne** hvordan **treffraten** i flernivå hurtigbuffer påvirker den gjennomsnittlige aksessiden.
- Studenten skal forstå hvorfor **settassosiative** og **fullassosiative hurtigbuffer** kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18).
- Studenten skal kjenne til hvordan man på programvarenivå kan påvirke **treffraten** i **hurtigbuffer**.

Læringsutbytte T6.1

- Studenten skal kunne forklare begrepene **lokalitet i tid** og **lokalitet i rom** og vite hvorfor lokalitet oppstår.
- Studenten skal forstå hvordan og hvorfor et **hierarki av minner** kan gi illusjonen av ett stort og raskt minne.
- Studenten skal forstå hvordan vi **konstruerer et direkteilordnet hurtigbuffer** (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord.
- Studenten kunne forklare hvordan **hurtigbuffer** kan **integreres** i en **samlebåndsarkitektur**.
- Studenten skal kjenne til prinsippene for hvordan hurtigbufferer håndterer **skriveoperasjoner**.
- Studenten skal kunne beregne hvordan minneaksesstid påvirker datamaskinens ytelse.
- Studenten skal kunne beregne hvordan treffraten i flernivå hurtigbuffer påvirker den gjennomsnittlige aksesstiden.
- Studenten skal forstå hvorfor settassosiative og fullassosiative hurtigbufferer kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18).
- Studenten skal kjenne til hvordan man på programvarenivå kan påvirke treffraten i hurtigbufferer.

Vi vil ha et stort og raskt minne

- ... men det går ikke – vi kan ikke få i pose og sekk
 - **Ytelse:** Mindre minner er typisk raskere enn store minner
 - **Kostnad:** Raske minneteknologier tar mer plass og koster dermed mer enn trege minneteknologier
- **Løsning: Minnehierarki**
 - Så lenge vi stort sett aksesserer det raske minnet, skaper vi illusjonen av et stort og raskt minne

Speed	Processor	Size	Cost (\$/bit)	Current technology
Fastest	Memory	Smallest	Highest	SRAM
	Memory			DRAM
Slowest	Memory	Biggest	Lowest	Magnetic disk or Flash memory

Menti!

Eksempel: Aksesstidsberegning



Lokalitetsprinsippet

- Observasjon: Et program aksesserer relativt få dataelementer i et kort tidsrom
- Lokalitet i tid:
 - «Temporal locality»
 - Hvis et program bruker et dataelement, er det sannsynlig at det vil bruke det igjen snart
- Lokalitet i (adresse)rom:
 - «Spatial locality»
 - Hvis et program bruker et dataelement, er det sannsynlig at det vil bruke et element på en adresse i nærheten snart

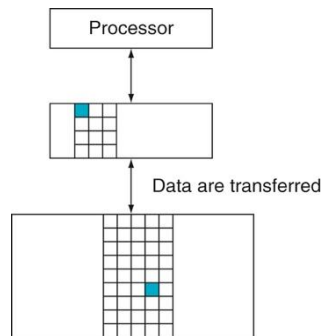
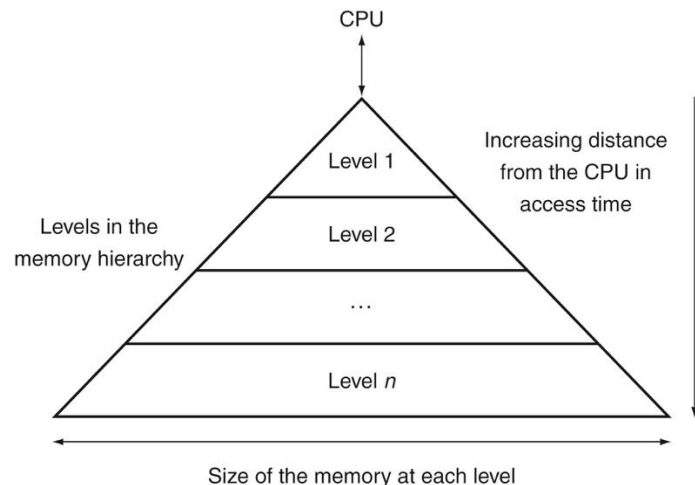
```
1  #include <stdio.h>
2
3  int main(){
4      int tabell[5];
5
6      for(int i=0;i<5;i++){
7          tabell[i] = i;
8          printf("Tabellverdien på adresse %p er %d\n",
9                &(tabell[i]),
10               tabell[i]);
11      }
12
13      return 0;
14 }
```

Vi kjører litt kode!

Lokalitet oppstår som en konsekvens av hvordan vi programmerer!

Minnehierarki

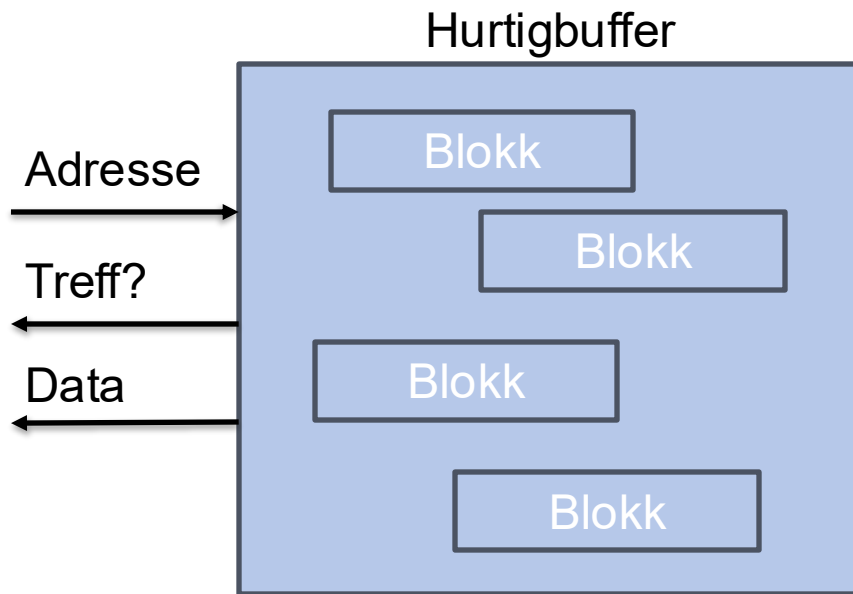
- Vi utnytter lokalitetsprinsippet til å skape illusjonen av et stort og raskt minne
- Hvordan vet vi hva vi skal lagre i det minste minnet (nivå 1)?
 - Vi flytter dataelementene nært prosessoren når de blir aksessert
 - Vi dataelementer vi ikke har brukt på en stund når vi trenger mer plass



Menti!

Hurtigbuffer («Cache»)

- En maskinvarestyrt minneenhet som kan lagre n dataelementer
 - kalt blokker eller linjer
- Vi trenger en effektiv maskinvareløsning for å finne ut om et dataelement er lagret i hurtigbufferet eller ikke
 - Hvis ja (treff), hent det!
 - Hvis nei (bom), send forespørselen videre til neste nivå
 - «Cache hit» og «Cache miss»



Hvordan lager vi et 16B hurtigbuffer?

Adresseformat:
(Anta 8 bit adresser)

7	6	5	4	3	2	1	0



Eksempel: Hurtigbuffer (8 bit adresser)

Les adresse 24 (0001 1000)

Adresseformat

Tag				Indeks		Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (16 byte)

Data		Tag	
Indeks	Verdi	Indeks	Verdi
3 (11)		3 (11)	
2 (10)		2 (10)	
1 (01)		1 (01)	
0 (00)		0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (desimal)	Adresse (binær)	Verdi (32 bit)
252	1111 11 00	-
...	...	...
28	0001 11 00	-
24	0001 10 00	375
20	0001 01 00	-
16	0001 00 00	-
12	0000 11 00	258
8	0000 10 00	42
4	0000 01 00	-
0	0000 00 00	-

Alltid '00' fordi vi henter 32 bit (4 byte) ord

Eksempel: Hurtigbuffer (8 bit adresser)

Les adresse 12 (0000 1100)

Adresseformat

Tag				Indeks		Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (16 byte)

Data		Tag	
Indeks	Verdi	Indeks	Verdi
3 (11)		3 (11)	
2 (10)	375	2 (10)	0001
1 (01)		1 (01)	
0 (00)		0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (desimal)	Adresse (binær)	Verdi (32 bit)
252	1111 11 00	-
...	...	...
28	0001 11 00	-
24	0001 10 00	375
20	0001 01 00	-
16	0001 00 00	-
12	0000 11 00	258
8	0000 10 00	42
4	0000 01 00	-
0	0000 00 00	-

Alltid '00' fordi vi henter 32 bit (4 byte) ord

Eksempel: Hurtigbuffer (8 bit adresser)

Les adresse 24 (0001 1000)

Adresseformat

Tag				Indeks		Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (16 byte)

Data		Tag	
Indeks	Verdi	Indeks	Verdi
3 (11)	258	3 (11)	0000
2 (10)	375	2 (10)	0001
1 (01)		1 (01)	
0 (00)		0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (desimal)	Adresse (binær)	Verdi (32 bit)
252	1111 11 00	-
...	...	...
28	0001 11 00	-
24	0001 10 00	375
20	0001 01 00	-
16	0001 00 00	-
12	0000 11 00	258
8	0000 10 00	42
4	0000 01 00	-
0	0000 00 00	-

Alltid '00' fordi vi henter 32 bit (4 byte) ord

Eksempel: Hurtigbuffer (8 bit adresser)

Les adresse 8 (0000 1000)

Adresseformat

Tag				Indeks		Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (16 byte)

Data		Tag	
Indeks	Verdi	Indeks	Verdi
3 (11)	258	3 (11)	0000
2 (10)	375	2 (10)	0001
1 (01)		1 (01)	
0 (00)		0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (desimal)	Adresse (binær)	Verdi (32 bit)
252	1111 11 00	-
...	...	...
28	0001 11 00	-
24	0001 10 00	375
20	0001 01 00	-
16	0001 00 00	-
12	0000 11 00	258
8	0000 10 00	42
4	0000 01 00	-
0	0000 00 00	-

Alltid '00' fordi vi henter 32 bit (4 byte) ord

Eksempel: Hurtigbuffer (8 bit adresser)

Adresse 8 (0000 1000) har blitt lest

Adresseformat

Tag				Indeks		Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (16 byte)

Data		Tag	
Indeks	Verdi	Indeks	Verdi
3 (11)	258	3 (11)	0000
2 (10)	42	2 (10)	0000
1 (01)		1 (01)	
0 (00)		0 (00)	

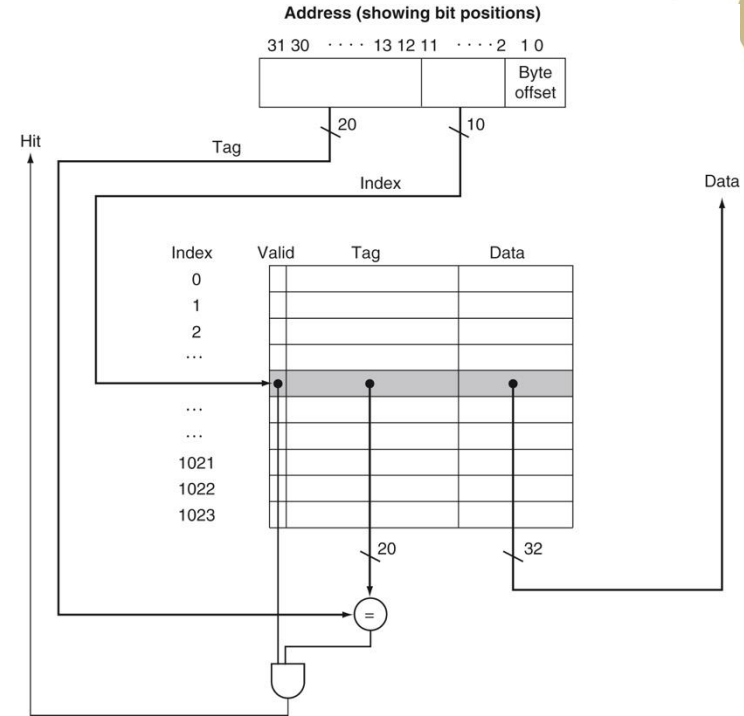
Minne ($2^8 = 256$ byte)

Adresse (desimal)	Adresse (binær)	Verdi (32 bit)
252	1111 11 00	-
...	...	...
28	0001 11 00	-
24	0001 10 00	375
20	0001 01 00	-
16	0001 00 00	-
12	0000 11 00	258
8	0000 10 00	42
4	0000 01 00	-
0	0000 00 00	-

Alltid '00' fordi vi henter 32 bit (4 byte) ord

Et direkteilordnet 4 KiB hurtigbuffer

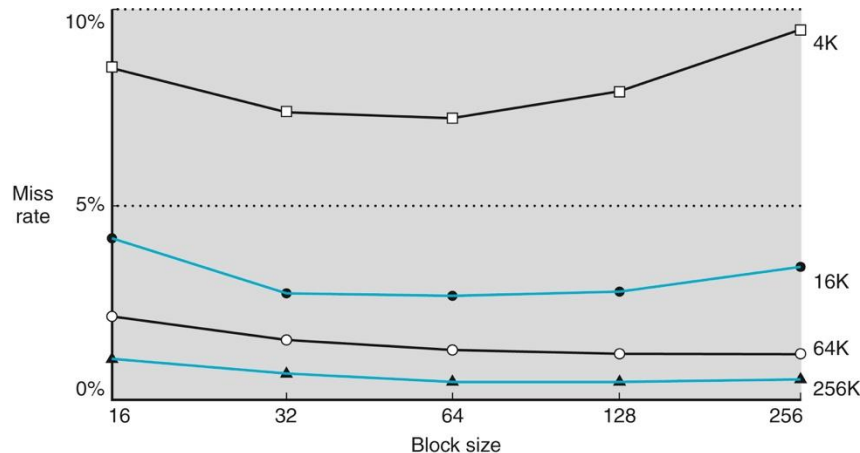
- Vi bruker bit 11-2 til indeks fordi sekvensiell aksess er vanlig
 - 10 adressebit gir $2^{10} = 1024$ indeksverdier
 - Hver hurtigbufferblokk lagrer 4 byte (32 bit) og hurtigbufferstørrelsen blir dermed $4 \times 2^{10} = 4096$ byte (4 KiB)
- Hver blokk må ha et «valid» bit
 - Valid = 1: Verdien er gyldig
 - Vi trenger disse ved oppstart og hvis vi må tømme hurtigbufferet



Figur 5.10

Hvor stor bør en blokk være?

- En hurtigbufferblokk kan være større enn ordbredden til prosessoren
- Større blokker gir:
 - Fordel: Bedre utnyttelse av lokalitet i rom
 - Ulempe: Plass til færre blokker i hurtigbufferet
 - Ulempe: Kostnaden ved å hente nye data øker

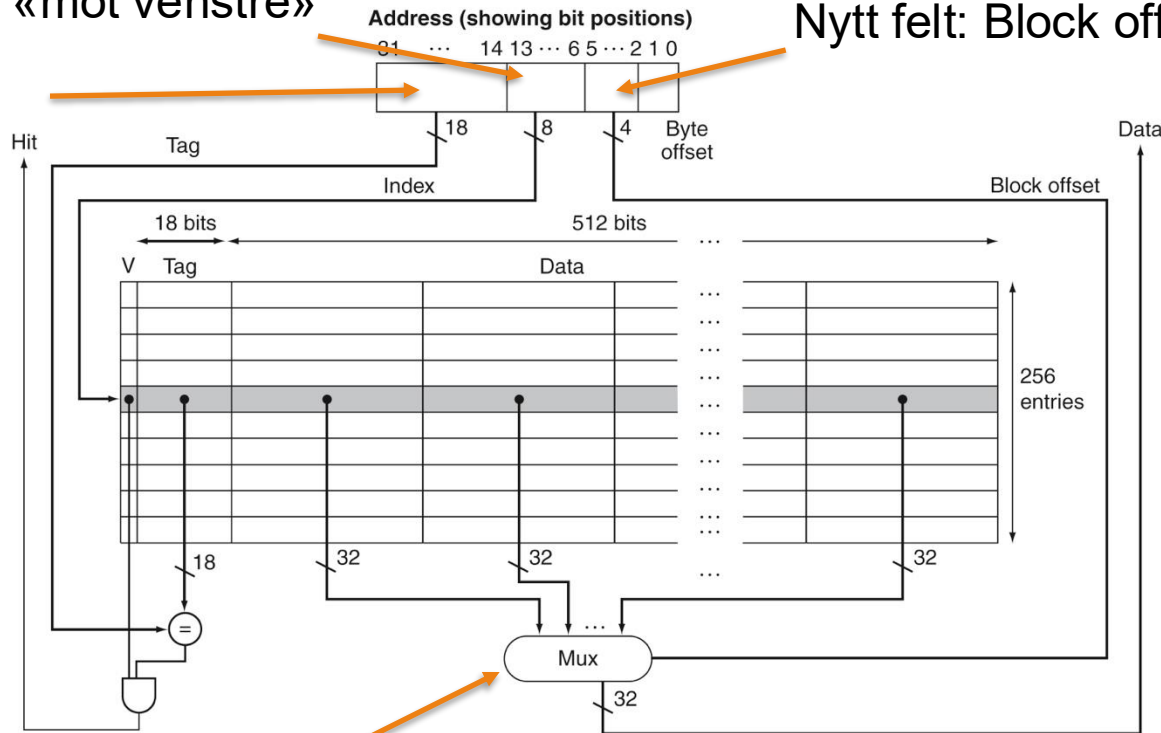


Flere ord i en blokk

Index er flyttet «mot venstre»

Færre bit i tag

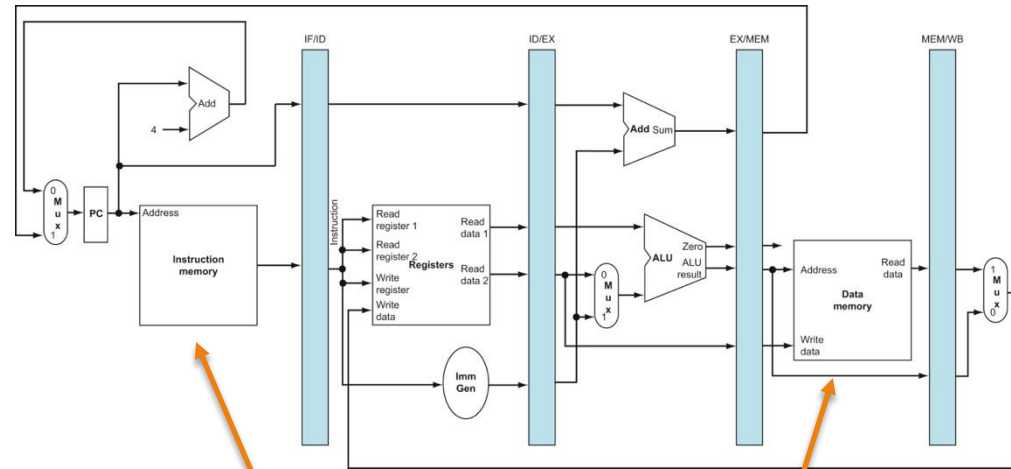
Nytt felt: Block offset



Vi trenger en multiplekser for å hente rett ord

Hvordan integrerer vi hurtigbufferne i samlebåndsprosessoren?

- Treff er lett: Vi gir hurtigbufferet en adresse og får instruksjon/data tilbake i samme sykel
 - ... hvis hurtigbufferet er lite nok
- Bom må håndteres
 - Det enkleste er å fryse samlebåndet til vi har fått hentet den manglende hurtigbufferblokken fra hovedminnet



Instruksjonshurtigbuffer

Datashurtigbuffer

Hvordan håndtere skriveoperasjoner?



- Når vi skriver til hurtigbufferet, må vi også oppdatere minnet
 - ... men dette må ikke skje med en gang!
- To skriveprinsipper:
 - «Write-through»: Oppdatere minnet hver gang blokka skrives
 - «Write back»: Oppdatere minnet når blokka kastes ut av hurtigbufferet
 - Da må vi holde styr på om en hurtigbufferblokk har blitt skrevet til («dirty bit»)
- Hva skjer når vi bommer på en skriveoperasjon?
 - «No write allocate»: Vi skriver rett til minnet
 - «Write allocate»: Vi henter blokka fra minnet før vi skriver til den
- Vi vil ofte legge til buffere som lagrer nylig skrevne data frem til vi kan oppdatere hurtigbufferet (typisk kalt «write buffer»)



Tema 6.2: Minneteknologier (Kapittel 5.2 og A.9)

MINNETEKNOLOGIER

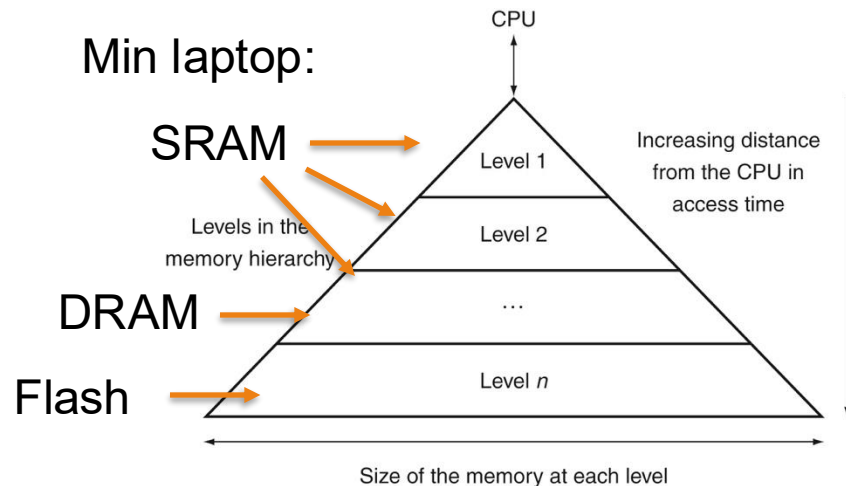
Læringsutbytte T6.2

- Studenten skal kunne forklare begrepene **volatilt og ikke-volatilt minne** samt **statisk og dynamisk minne**.
- Studenten skal kjenne til omtrentlig **aksesstid** og **kostnad per bit** for **SRAM**, **DRAM**, **Flash**, og **magnetisk disk** og kunne forklare hvordan dette påvirker hvordan teknologiene brukes i minnehierarkiet.
- Studenten skal kunne **konstruere** et **SRAM minne** fra grunnleggende kombinatoriske og sekvensielle komponenter (kunne gjengi og forklare figur A.9.3).
- Studenten skal kunne beskrive **prinsippene** for hvordan vi konstruerer **DRAM minner** (kunne gjengi og forklare figur A.9.5 og A.9.6).

Volatile og ikke-volatile minner

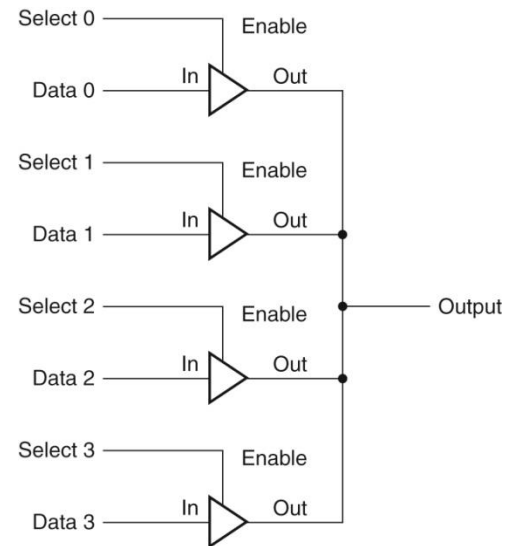
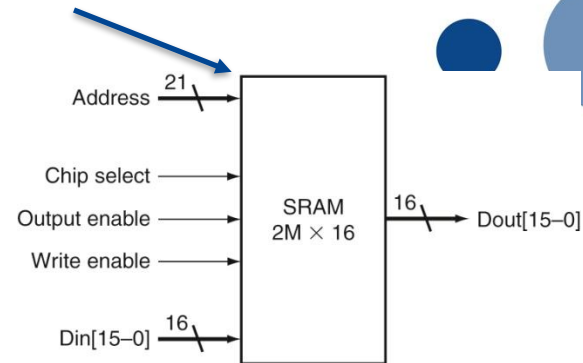
- Volatile minnetyper mister verdiene når vi skruer av strømmen
 - SRAM: «Static Random Access Memory»
 - DRAM: «Dynamic Random Access Memory»
- Ikke-volatile minnetyper beholder verdiene når vi skruer av strømmen
 - Flash
 - Magnetiske harddisker

Memory technology	Typical access time	\$ per GiB in 2020
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$3–\$6
Flash semiconductor memory	5,000–50,000 ns	\$0.06–\$0.12
Magnetic disk	5,000,000–20,000,000 ns	\$0.01–\$0.02



SRAM (1)

- «Random Access Memory»
 - ... betyr at det tar like lang tid å aksessere alle adresser
- «Static RAM»
 - Lagrer verdiene i låser
 - Verdiene beholdes så lenge strømmen er på (statisk)
- Det var da veldig likt en registerfil?
 - ... men registerfila skalerer ikke fordi den bruker en multiplekser per utgang
 - En 32-til-1 multiplekser er OK, men en 2097152-til-1 multiplekser er ikke OK (som vi trenger til minnet til høyre)
- Løsning: «Tristate buffer»
 - Slipper enten inngangsverdien igjennom til utgangen eller kobler seg fra utgangen («high impedance state»)
 - Gjør det mulig å koble flere minneelementer til samme utgang!



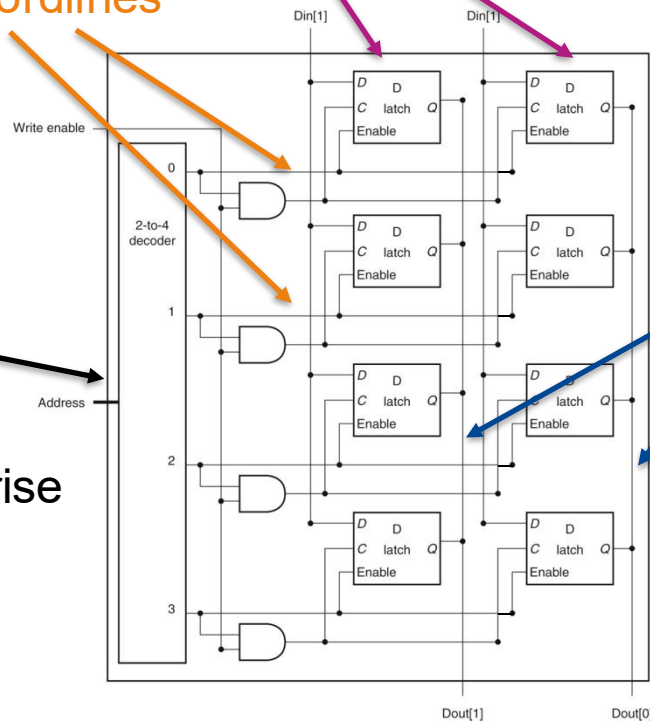
SRAM (2)

Alle låser har tristate buffer på Q-utgangen

... men blir ikke dekoderen også stor?

Løsning: Organiser minnet som en matrise

Wordlines

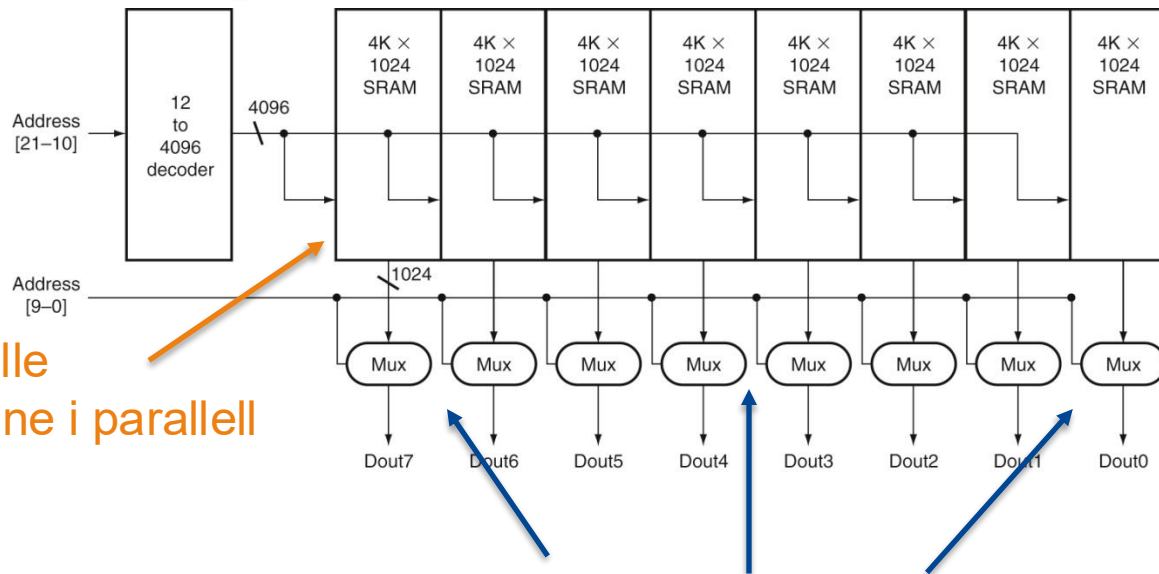


Bitlines

Eksempel: En 4x2 SRAM (Figur A.9.3)

SRAM (3)

Dekoderen tar seg av noen av adressebitene



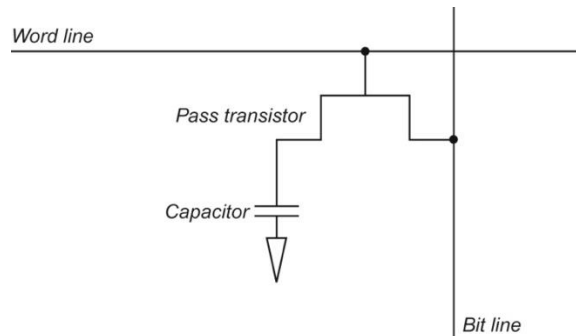
Vi leser fra alle minneblokkene i parallell

Eksempel:
En 4Mi x 8 SRAM

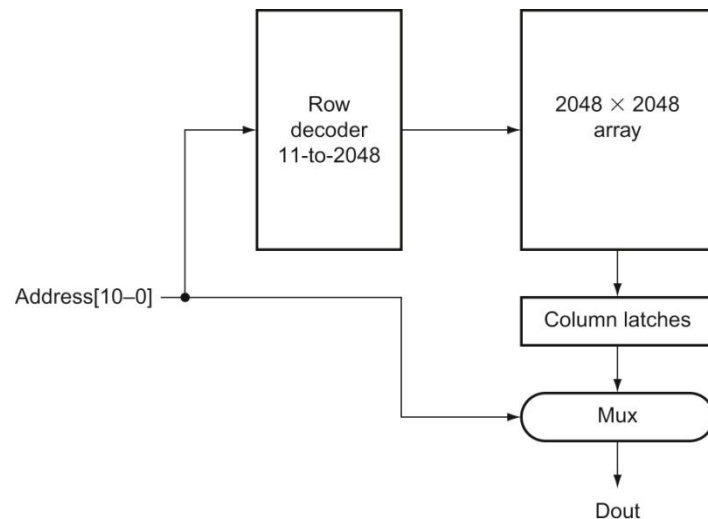
Hver minneblokk leverer ett bit som velges ut av multipleksere basert på resten av adressen

DRAM

- DRAM lagrer verdier i kondensatorer
 - Høyere tetthet enn SRAM og dermed billigere per bit
 - ... men mister ladningen over tid (dynamisk)
 - Løsning: Lese verdiene ut og skrive de tilbake før de forsvinner («refresh»)
- Kondensatoren lades ut når vi leser
 - ... og derfor må vi ha låser på utgangen og skrive raden tilbake igjen etterpå
- Adressen sendes i to skritt:
 - Adressen til raden («Row Access Strobe (RAS)»)
 - Adressen til kolonnen («Column Access Strobe (CAS)»)
 - Vi kan aksessere neste verdi raskt ved å bare sende en ny kolonneadresse!



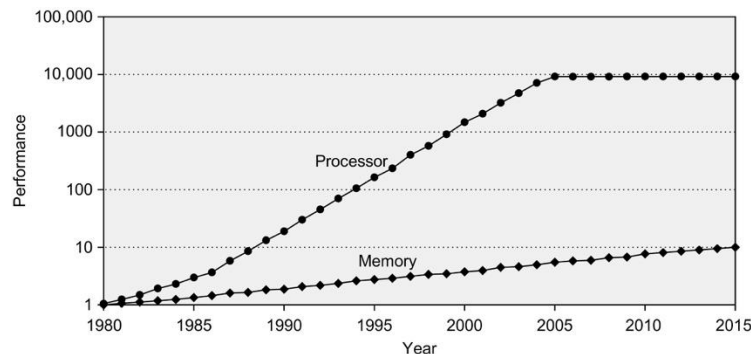
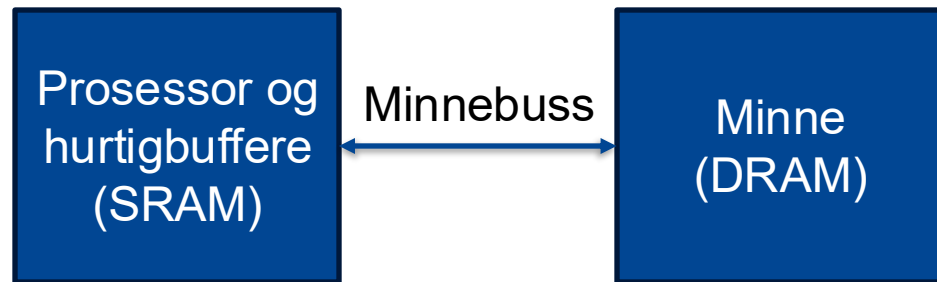
Figur A.9.5



Figur A.9.6

Minnebuss

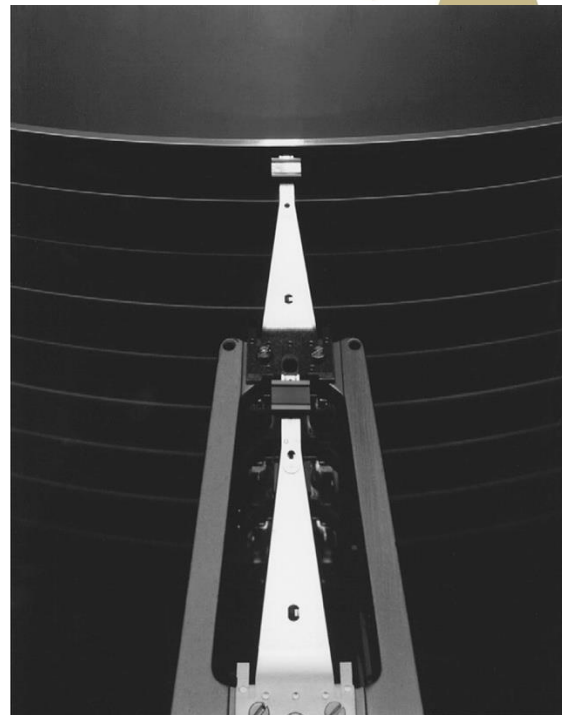
- SRAM/logikk bruker ulike produksjonsprosesser
 - SRAM/logikk-prosessen er optimalisert for raske transistorer
 - DRAM-prosessen er optimalisert for høy tetthet for kondensatorene
- Prosessor og hurtigbuffer er dermed typisk på en brikke mens DRAM er på andre brikker
 - Minnebussen kobler disse sammen og er designet for høy båndbredde
 - Eksempler: DDR5, GDDR5, HBM2, etc.
 - DDR = «Double Data Rate»: Sender data på både stigende og synkende klokkeflanke



«The Memory Wall»

Flash og magnetisk disk

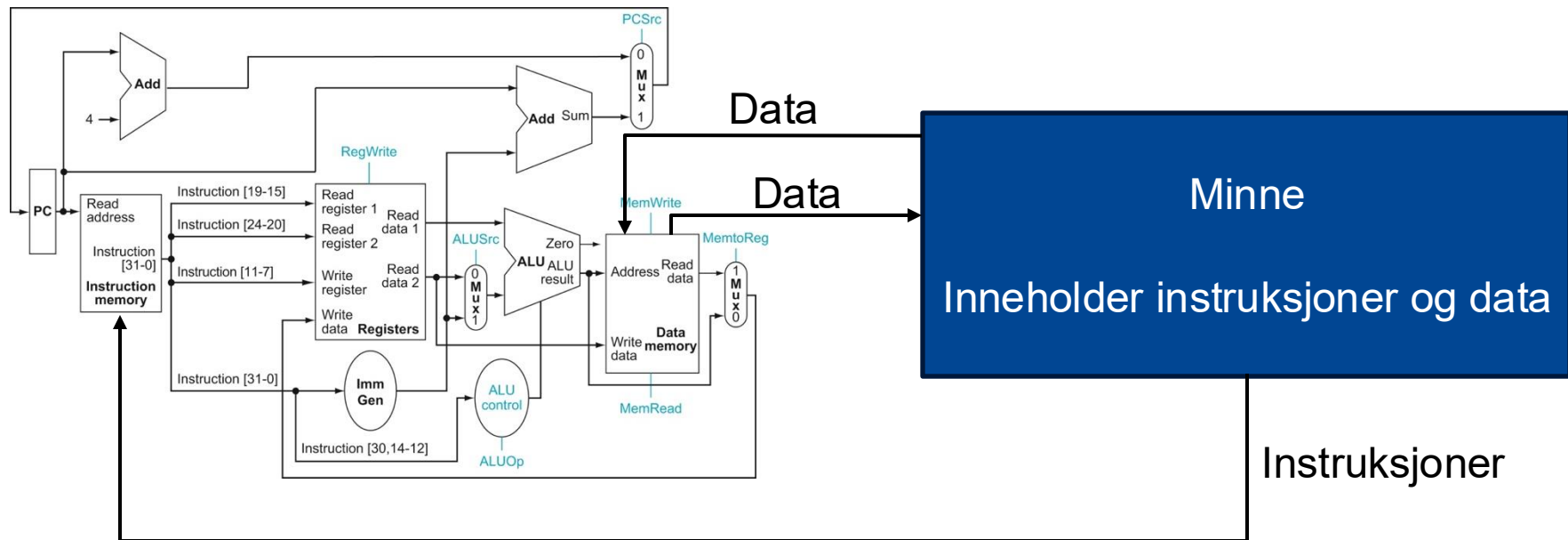
- Flash
 - En EEPROM («Electrically Erasable Programmable Read-Only Memory»)
 - Hvert bit kan bare skrives et gitt antall ganger før det blir utslitt
 - Løsning: Fordel skriveingen utover hele minnet («wear leveling»)
- Magnetisk disk
 - Hvert bit er lagret i et magnetisk materiale på en snurrende plate
 - Aksess krever at vi:
 - Flytter lesehodet til riktig spor
 - Venter til vi har rotert til riktig sektor
 - Gjennomføre lese eller skriveoperasjonen
 - Ikke «random access»: Det tar lengre tid å lese noen verdier enn andre
 - Det lønner seg å lese mye data når du først har fått lesehodet til riktig sted





OPPSUMMERING

Prinsippet om lagrede program



Prinsippet om lagrede program

Proessorbrikken

Ofte flere nivåer med hurtigbuffer (SRAM)

